DeepWeeds T03

Mimari ve Kod Analizi Raporu

Takim: BM_TAKIMKODU_T03

Tema: T03 - Tarim Teknolojileri

Surum: 0.1.0

Rapor Tarihi: 24 October 2026

En Iyi Model: EfficientNet-B3 (exp-effb3-sqrt)

Macro-F1 (Val): 0.8181 | Kalibreli: 0.8568

Bu rapor proje kodlarinin otomatik analizi ile olusturulmustur.

. ICINDEKILER

1 Pr
2 Si
3 M
3.1 config.py - Yapilandirma Yonetimi
3.2 data.py - Veri Yukleme ve Artirma
3.3 models.py - Model Olusturma
3.4 engine.py - Egitim Dongusu (Trainer)
3.5 inference.py - Cikarim ve Tahmin
3.6 metrics.py - Metrik Hesaplama
3.7 reports.py - Gorsel Raporlama
3.8 cli.py - Komut Satiri Arayuzu
3.9 api.py / streamlit_app.py - Web Arayuzleri
3.10 utils.py - Yardimci Fonksiyonlar
4 Ve
5 Eg
6 Se

1. PROJEYE GENEL BAKIS

DeepWeeds T03, tarla fotograflarindan 8 zararli yabanci ot turunu ve "Negative" (ot yok) sinifini ayirt eden uc tan uca bir goru siniflandirma sistemidir. Proje, Yarismaci icin hazirlanmis olup Macro-F1 metriğine gore optimize edilmistir.

Sistem uc ana bilesenden olusur:

- Egitim (train): Derin ogrenme modeli (EfficientNet-B3/ResNet50/ConvNeXt), ~12.000 etiketli tarla fotoğrafı ile egitilir. Early stopping, cosine LR, AMP, label smoothing ve sinif dengesizligi icin sqrt weighted loss kullanilir.
- Tahmin (predict): Egitilmis model, yarisma formatinda predictions.csv uretir. Validation setinde ogrenilen per-sinif kalibrasyon carpänlari test tahminlerine otomatik uygulanir.
- Demo/API (api, streamlit): FastAPI tabanli REST servisi ve Streamlit tabanli goruntu yukleme arayuzu. Her ikisi de internet baglantisi gerektirmez, yerel calisir.

Hedef siniflar (9 sinif): Chinee apple, Lantana, Parkinsonia, Parthenium, Prickly acacia, Rubber vine, Siam weed, Snake weed, Negative.

2. SISTEM MIMARISI

Proje modüler, katmanli bir mimariye sahiptir. Her modul tek bir sorumluluga odaklanir (Single Responsibility Principle). Bagimlilik yonunde veri akisi asagidaki gibidir:

```
Config (config.py)
    |
    v
Data (data.py)  -->  DataLoader (train/val/test)
    |
    v
Model (models.py)  -->  timm backbone + custom head
    |
    v
Engine (engine.py)  -->  Trainer (fit, validate, checkpoint)
    |
    v
Inference (inference.py)  -->  Predictor, TTA, benchmark
    |
    v
Metrics (metrics.py)  -->  Macro-F1, per-class, confusion matrix
    |
    v
Reports (reports.py)  -->  PNG grafikler, CSV, classification_report
    |
    v
CLI (cli.py)  -->  train / predict / evaluate / api / streamlit
```

Bu bagimsizlik sayesinde her bilesen ayrı test edilebilir, degistirilebilir ve yeniden kullanilabilir. Ornegin data.py degistirilmeden models.py yeni bir backbone ile guncellenebilir.

3. MODUL BAZLI KOD ANALIZI

3.1 config.py - Yapilandirma Yonetimi

Tum egitim ve veri parametrelerini merkezi bir yerden yoneten dataclass tabanli yapilandirma moduludur. Bu sayede "magic number"lar onlenir, deneyler yeniden uretilebilir hale getirilir.

Ana siniflar:

- DataConfig: veri yollari, image/label kolonlari, img_size, batch_size, num_workers, use_aug
- ModelConfig: model adi (timm), num_classes, pretrained bayragi
- TrainConfig: epochs, lr, weight_decay, label_smoothing, balance stratejisi, patience, seed, amp, deterministic, run_root, run_name
- build_label_map(): etiket stringlerini 0-8 arasi integer ID'lere cevirir
- save_json(): dataclasslari JSON olarak kaydeder (config.json -> yeniden uretim icin kritik)

balance parametresi 4 modu destekler:

- none: agirlik yok, standart CrossEntropy
- weighted_loss: sinif frekanslarinin tersi (1/freq) agirlik
- sqrt_weighted_loss: karekok agirligi sqrt(1/freq) - EN IYI SONUÇ
- balanced_sampler: WeightedRandomSampler ile batch seviyesinde dengeleme

3.2 data.py - Veri Yukleme ve Artirma

DeepWeedsData sinifi veri yukleme, bolme, artirma (augmentation) ve DataLoader olusturma sorumlulugunu ustlenir.

Kisas yontemler ve ozellikler:

- __init__: CSV oku, train/val/test bol (stratified 80/10/10), class_counts hesapla
- get_dataloaders(): train/val/test icin DataLoader dondurur
- train_transform: RandomResizedCrop(0.6-1.0) + HorizontalFlip + ColorJitter + Normalize(ImageNet) + opsiyonel RandAugment (strong_aug=True)
- val/test_transform: Resize + CenterCrop + Normalize (deterministik)
- class_counts, class_names, num_classes property'leri
- id_series(): yarisma formatinda ID kolonu (image_id veya index)

Veri bolme stratejisi: train.csv icindeki tum verinin %80'i train, %10'u val, %10'u test. Stratified bolme sayesinde her sinif oranlari korunur. test.csv asla egitime karismaz.

3.3 models.py - Model Olusturma

timm kutuphanesini wrapper ederek herhangi bir ImageNet on-eğitilmis modeli 9 sinifli siniflandirmaya uyarlayan moduldur.

Kisas fonksiyonlar:

- build_model(ModelConfig): timm.create_model ile backbone olustur, son katmani nn.Linear(in_features, 9) ile degistir, pretrained agirliklari yukle
- count_parameters(model): egitilebilir parametre sayisini hesapla (Milyon cinsinden loglar)
- SUGGESTED_MODELS: onerilen backbone listesi

```
SUGGESTED_MODELS = [
    'resnet50',           # ~25M param, hizli, guvenilir
    'efficientnet_b3',    # ~12M param, EN IYI F1 (0.818)
    'convnext_tiny',      # ~28M param, modern mimari
    'vit_small_patch16_224', # ViT, daha cok veri ister
    'swin_tiny_patch4_window7_224', # Hierarchical ViT
]
```

Model secimi: EfficientNet-B3, parametre sayisi/dengesi ve Macro-F1 performansi icin secilmistir. img_size=256 ile birlikte strong augmentation kullanildiginda en iyi sonucu verir.

3.4 engine.py - Egitim Dongusu (Trainer)

Trainer sinifi egitimin tum dongusunu yonetir: forward/backward, optimizer/scheduler, AMP, early stopping, checkpoint kaydetme, logging.

Ana bileşenler:

- __init__: model, data, config, device, run_dir alir. Optimizer (AdamW), Scheduler (CosineAnnealingLR), Criterion (CrossEntropy + label_smooth + class_weights) kurar
- compute_class_weights(): balance moduna gore agirliklari hesaplar
- sqrt_weighted_loss icin: weight = sqrt(1 / class_freq) -> normalize
- train_epoch(): train loader uzerinde AMP ile forward/backward, grad clip (max_norm=1.0)
- validate(): val loader uzerinde macro-F1 hesaplar (sklearn f1_score, average="macro")
- fit(): epoch dongusu, her epochta validate, en iyi macro_F1'de best.pt kaydet, early stopping (patience), history.csv yaz

Checkpoint icerigi: model_state_dict, optimizer_state_dict, scheduler_state_dict, epoch, best_f1, config, class_names. Bu sayede eğitim kesintisiz devam ettirilebilir.

3.5 inference.py - Cikarim ve Tahmin

Predictor sinifi egitilmis modeli yukleyip tekil/batch tahmin, benchmark ve TTA saglar. CLI, API ve Streamlit tum bu sinifi kullanir (DRY prensibi).

Kisas yontemler:

- from_checkpoint(path, device): checkpoint yukle, modeli eval mode'a al, normalizasyon parametrelerini (mean/std) al
- predict_pil(PIL.Image): tek goruntu icin olasilik vektoru dondur (9 sinif)
- predict_df(DataFrame): CSV'deki tum goruntuleri isle, y_true (varsa) ve y_prob dondur

- predict_tta(PIL.Image): 5 augmentasyon (orjinal, hflip, vflip, rotate90, rotate270) ile tahmin alip ortalama alir - daha karalli, ~5x yavas
- benchmark_latency(df, n): n ornek uzerinde ortalama cikarim suresini olcer (ms)
- write_predictions_csv(): yarisma formatinda CSV uret (id, class_0..class_8, prediction, latency_ms, model_size_bytes). Kalibrasyon carpânlari varsa uygular.

3.6 metrics.py - Metrik Hesaplama

sklearn.metrics wrapper'i, Macro-F1 odakli metrikler uretir. ComputeMetrics sonucu bir dataclass (MetricsResult) olarak dondurur.

Hesaplanan metrikler:

- macro_f1: f1_score(average="macro") - Ana basari kriteri
- accuracy: accuracy_score
- per_class: dict[sinif_adi -> {precision, recall, f1, support}]
- confusion_matrix: nxn normalize matris (row=true, col=pred)

Macro-F1, sinif dengesizligine karsi dayaniklidir cunku her sinifi esit agirliklar. Negative sinifi (1277 ornek) ile Snake weed (50 ornek) ayni etkiye sahiptir.

3.7 reports.py - Gorsel Raporlama

Egitim sirasindaki ve sonundaki tum gorsellestirmeleri uretir. Matplotlib/Seaborn kullanir. Tum grafikler runs/<run>/reports/ altina kaydedilir.

- save_training_curves(): Loss (train/val), Macro-F1 (val), LR egrileri - 3 alt grafik
- save_confusion_matrix(): Normalize confusion matrix heatmap (annotated)
- save_per_class_f1(): Sinif bazinda F1 cubuk grafigi (renk kodlu)
- save_class_distribution(): Train vs Val sinif dagilimi karsilastirmasi
- save_misclassified_grid(): Yanlis siniflandirilan orneklerin grid gorseli (en cok 16 adet)
- save_classification_report(): sklearn classification_report metin dosyasi
- save_all(): Yukaridakilerin hepsini tek cagirida calistirir, metrics.json da yazar

3.8 cli.py - Komut Satiri Arayuzu

argparse tabanli ana giris noktasi. 5 alt komutu (subcommand) yonetir. Her komut icin varsayilan degerler, tip kontrolu ve yardim metni tanimlanmistir.

Komutlar:

- train: Model egitir, runs/<run_name>/ altina ciktilari yazar
- predict: Val/test icin predictions.csv uretir (kalibrasyon otomatik)
- evaluate: Gercek etiket + tahmin CSV'lerinden metrik/rapor uretir
- api: FastAPI + Uvicorn baslatir (port 8000, Swagger /docs)
- streamlit: Streamlit arayuzu baslatir (port 8501)

train icin onemli varsayilanlar: model=resnet50, epochs=25, lr=1e-4, balance=weighted_loss, img_size=224, batch_size=32, aug=True, amp=True. En iyi sonuc icin: --model efficientnet_b3 --img-size 256 --balance sqrt_weighted_loss --lr 5e-5 --epochs 20 --patience 6 --aug

3.9 api.py ve streamlit_app.py - Web Arayuzleri

api.py (FastAPI):

- create_app(model_path, device): FastAPI app olusturur
- GET /: Basit HTML test sayfasi (file upload form)
- POST /predict: multipart/form-data file kabul eder, JSON dondurur

- Swagger UI: /docs endpoint'i otomatik olusur

streamlit_app.py (OOP tasarim):

- ModelService (Singleton): Model yuklemeyi cache'ler, tek yukleme garantiler
- InferenceEngine: Normal ve TTA modlarini yonetir, benchmark yapar
- StreamlitUI: Header, sidebar, uploader, preview, results, footer bileşenleri
- StreamlitApp (Facade): Butun bilesenleri birlestirir, run() dongusunu yonetir
- Ozellikler: Drag-drop yukleme, TTA ac/kapa, olasilik cubuk grafigi, bar chart, model bilgileri (cihaz, boyut, checkpoint), model yeniden yukleme butonu

3.10 utils.py - Yardimci Fonksiyonlar

Proje geneli yardimci fonksiyonlar:

- get_device(): cuda varsa cuda, yoksa cpu dondurur
- set_seed(seed, deterministic): random, numpy, torch, cuda seed'leri ayarlar
- get_logger(): Rich formatli konsol logger (seviye, zaman, modul adi)
- Kod tekrarini onler, deterministikligi saglar.

4. VERI AKISI VE PIPELINE

Egitim Pipeline'i:

```
1. config.py -> DataConfig, ModelConfig, TrainConfig olustur
2. DeepWeedsData(config) -> CSV oku, stratify bol, transform hazirla
3. DataLoader(train/val) -> batch, shuffle, num_workers
4. build_model(ModelConfig) -> timm backbone + 9 sinif head
5. Trainer(model, data, config) -> optimizer, scheduler, criterion
6. for epoch in epochs:
     train_epoch()  -> forward, loss, backward, step, LR step
     validate()     -> macro_F1 hesapla
     if best_F1:    -> best.pt kaydet
     if no_improve: -> early stop
7. history.csv, config.json, best.pt, calibration.json kaydet
8. reports/ -> tum grafikler
```

Tahmin (Inference) Pipeline'i:

```
1. Predictor.from_checkpoint(best.pt) -> model yukle, eval()
2. DeepWeedsData -> test DataLoader (val transform)
3. for batch in loader:
     with AMP: probs = model(images).softmax(1)
4. Kalibrasyon carpânlari (calibration.json) varsa: probs *= multipliers
5. argmax -> prediction sinifi
6. write_predictions_csv() -> predictions.csv (yarisma format)
```

Kalibrasyon Mekanizmasi:

Validation setinde model ciktilari (logits/probs) ile gercek etiketler karsilastirilip her sinif icin bir carpân (multiplier) ogrenilir. Bu carpânlar test tahminlerinde olasiliklar ile carpilir. Bu islem "Temperature Scaling"in per-sinif basit versiyonudur ve Macro-F1'de ~0.04 (4 puan) artis saglar.

5. EGITIM STRATEJILERI VE HIPERPARAMETRELER

Denenen stratejiler ve sonuclari (sweep_results.json'dan):

Model	Strateji	img_size	Macro-F1	Kalibreli F1	Status
ResNet50	weighted_loss	224	0.7300	0.7873	Baseline
EffNet-B3	sqrt_weighted_loss	256	0.8181	0.8568	BEST

Basarili olan stratejilerin ortak ozellikleri:

- sqrt_weighted_loss: Karekok agirlik, asiri agirliklandi siniflari (Negative) baskilamaz, az ornekli siniflari (Snake weed) yeterince onpluler
- Strong Augmentation (RandAugment): Generalizasyonu artirir, overfitting onler
- img_size=256: EfficientNet-B3 icin onerilen cozunurluk, daha fazla ozellik yakalar
- lr=5e-5: On-eğitilmis agirliklarin bozulmamasi icin dusuk learning rate
- Label Smoothing (0.1): Overconfidence onler, kalibrasiyon iyilestirir
- AMP (Mixed Precision): ~2x hizlanma, VRAM tasarrufu

- Cosine LR + Early Stopping: En iyi epochta durma, overfit onleme

6. SONUCLAR VE PERFORMANS

En iyi model: exp-effb3-sqrt (EfficientNet-B3, sqrt_weighted_loss, strong_aug, img=256)

6.1 Sinif Bazinda Performans (Kalibreli Validation)

Sinif	Precision	Recall	F1-Score	Support
Parkinsonia	0.935	0.949	0.942	316
Parthenium	0.886	0.909	0.897	197
Siam weed	0.982	0.834	0.902	193
Lantana	0.853	0.924	0.887	238
Rubber vine	0.797	0.984	0.881	184
Negative	0.944	0.794	0.863	1277
Prickly acacia	0.597	0.938	0.729	161
Chinee apple	0.583	0.761	0.660	46
Snake weed	0.447	0.920	0.601	50

Gozlemler:

- Parkinsonia, Parthenium, Siam weed, Lantana, Rubber vine: F1 > 0.88 (cok iyi)
- Negative: Recall dusuk (0.79) cunku bazi otlu goruntuler Negative olarak tahmin ediliyor
- Prickly acacia, Chinee apple, Snake weed: Dusuk support (<200) nedeniyle zorlu siniflar
- Snake weed: Precision cok dusuk (0.45), diger siniflarla karisikligi yuksek

Genel Metrikler:

- Macro-F1 (raw): 0.8181
- Macro-F1 (kalibreli): 0.8568 (+4.87 puan artis)
- Accuracy: 0.8591
- Egitim suresi: ~40 dakika (RTX 3060 12GB), ~2378 saniye
- Model boyutu: ~48 MB (EfficientNet-B3)
- Cikarim suresi: ~15-25 ms/goruntu (GPU), ~100-200 ms (CPU)

. KAYNAKLAR VE REFERANSLAR

- DeepWeeds Dataset: Olsen et al., "DeepWeeds: A Multiclass Weed Species Image Dataset", 2019
- timm: Ross Wightman, "PyTorch Image Models", https://github.com/huggingface/pytorch-image-models
- EfficientNet: Tan & Le, "EfficientNet: Rethinking Model Scaling for CNNs", ICML 2019
- Label Smoothing: Szegedy et al., "Rethinking the Inception Architecture for CV", CVPR 2016
- Focal Loss / Class Weighting: Lin et al., "Focal Loss for Dense Object Detection", ICCV 2017
- Temperature Scaling: Guo et al., "On Calibration of Modern Neural Networks", ICML 2017
- RandAugment: Cubuk et al., "RandAugment: Practical Automated Data Augmentation", CVPR 2020
- Cosine Annealing: Loshchilov & Hutter, "SGDR: Stochastic Gradient Descent with Warm Restarts", ICLR 2017
- AdamW: Loshchilov & Hutter, "Decoupled Weight Decay Regularization", ICLR 2019
- Mixed Precision (AMP): Micikevicius et al., "Mixed Precision Training", ICLR 2018

RAPOR SONU